BSc Hons in Computing

2024.2

Problem Solving with Data Structures and Algorithms (PDSA-II)

Group Report: Minimum Cost Assignment Problem

Coursework 1

Leader Name: E M A S B EKANAYAKA
Cov. Index: 16110762
NIBM Index: COBSCCOMP242P-065

Member Name: M D S Y RANARAJA
Cov. Index: 16116133
NIBM Index: COBSCCOMP242P-075

Faculty of Engineering, Environment and Computing
School of Computing, Electronics and Mathematics
Coventry University

School of Computing and Engineering
National Institute of Business Management

List of Figures

List of Tables

Contents

Contribution Table

Table 1: Group Contribution and Marks

Index	**NIBM Index**	**Marks /10**	**Contribution**
16110762	COBSCCOMP242P-065	8	Backend – algorithms, database, API, unit tests
16116133	COBSCCOMP242P-075	7	Frontend – React UI, game screens, timing chart

1. Minimum Cost Problem

1.1. Functionality

1.1.1. Code Segment: N changes randomly from 50 to 100 in each game round

In our FastAPI backend, the number of workers and tasks N is picked randomly at the start of every round. We used Python's random.randint function to make sure N stays between 50 and 100.

Figure 1: Code sample showing how N is randomly generated between 50 and 100.

1.1.2. Code Segment: Cost of assigning each task to each employee changing randomly from $20 to $200

Once N is decided, the app makes an N x N cost matrix. We used another random.randint(20, 200) call to fill each cell in the matrix. This makes every game round unique and different for the users.

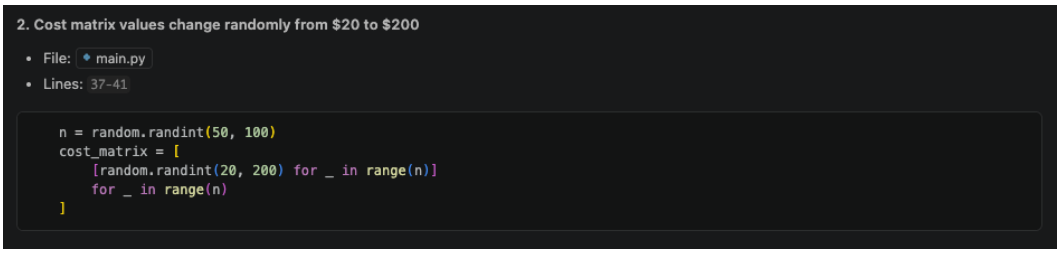

Figure 2: This code part shows how cost values are set randomly from $20 up to $200.

1.1.3. Code Segment: Record in the database how long it takes to find the minimum cost using each algorithm

The system runs both of the algorithms and records the time it took. We used time.perf_counter() to get high precision timing in milliseconds. All this data gets saved into the algorithm_results table in SQLite database.

Figure 3: Backend code that measures the speed of each algorithm.

1.1.4. Code Segment: Two different appropriate algorithms

The project uses two different ways to solve the assignment problem:

1. **Greedy Approach**: This is a fast $O(n^2)$ method. It doesn't always give the best answer but it is very quick.
2. **Hungarian Algorithm**: A more complex $O(n^3)$ method. This one is mathematically proven to always find the absolute minimum cost.

Figure 4: Our implementation of the Greedy logic.

- Hungarian algorithm implementation:
  - File: hungarian.py
  - Lines: 19-60

```python
def hungarian_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    ...
    # Step 1 — Row reduction
    for i in range(n):
        row_min = min(matrix[i])
        for j in range(n):
            matrix[i][j] -= row_min

    # Step 2 — Column reduction
    for j in range(n):
        col_min = min(matrix[i][j] for i in range(n))
        for i in range(n):
            matrix[i][j] -= col_min

    # Iterate until minimum line cover equals n
    while True:
        row_cover, col_cover = _min_line_cover(matrix, n)
        if len(row_cover) + len(col_cover) >= n:
            break
        _adjust_matrix(matrix, n, row_cover, col_cover)
```

Figure 5: The implementation of the Hungarian algorithm.

1.1.5. Code Segment: Save player's name and their response in the database

When people play the game in Manual or Mixed modes, they can type in their name. The system then saves their name and the total cost they calculated. This is linked to the actual algorithm results to compare later.

5. Save player name to the database

- File: main.py
- Lines: 120-124, 140-149

```python
    round_id = save_round(
        n,
        [{"algorithm_name": r.algorithm_name, "total_cost": r.total_cost, "time_ms": r.time_ms} for r in results],
        player_name=body.player_name or None,
        user_total_cost=user_total_cost,
    )
```

```python
@app.post("/api/rounds/update-player-name")
def update_rounds_player_name(body: UpdatePlayerNameRequest):
    """Update player_name for a list of rounds (used when game ends)."""
    if not body.round_ids:
        raise HTTPException(status_code=400, detail="round_ids cannot be empty")
    if not body.player_name or not body.player_name.strip():
        raise HTTPException(status_code=400, detail="player_name cannot be empty")

    update_player_name(body.round_ids, body.player_name.strip())
    return {"status": "ok", "updated_rounds": len(body.round_ids)}
```

Figure 6: Code that handles saving the player name to the DB.

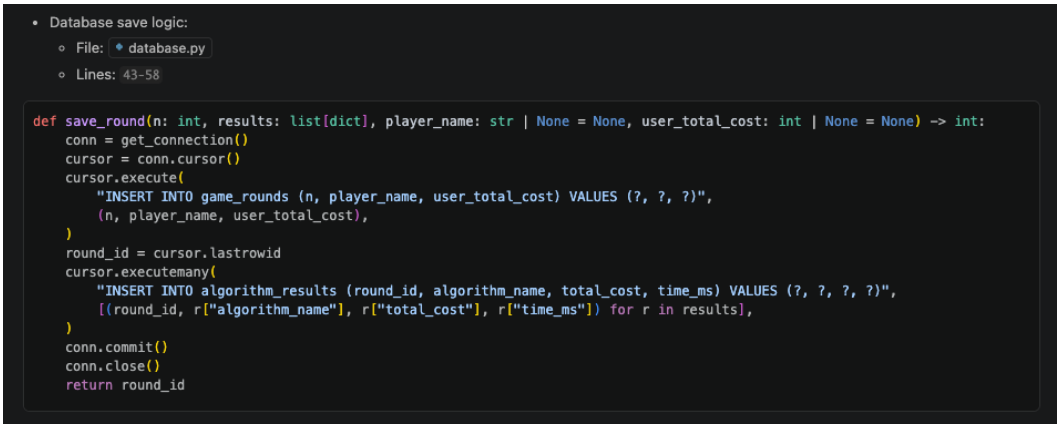

Figure 7: The backend function used to persist the round results.

1.1.6. Database Table structure for the Game

We used two main tables to keep the game data organized:

- `game_rounds`: This stores general round info like the value of N, player name, and the time the round happened.
- `algorithm_results`: This table is linked to the rounds table and stores the cost and time for each specific algorithm.

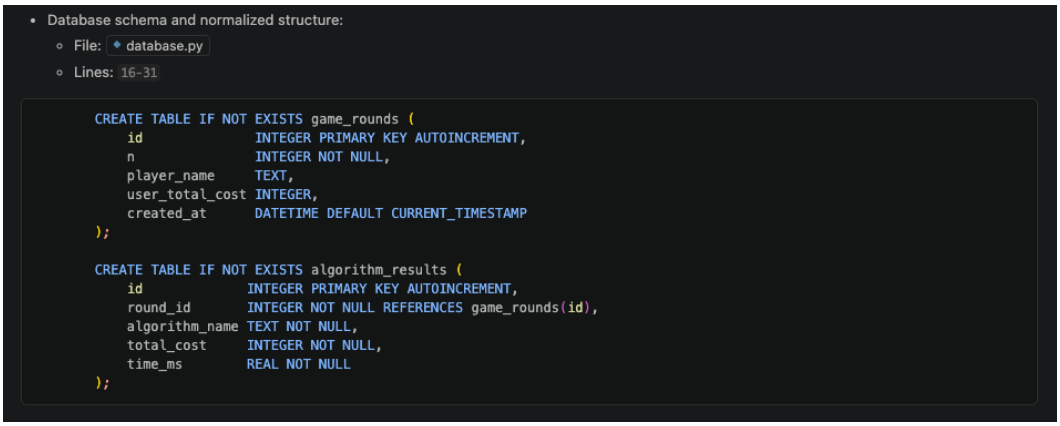

Figure 8: The SQLite database schema layout.

1.2. User Interface

1.2.1. UI screenshot for inputs and answers

The frontend was built using React and Vite. It has simple buttons for the users to choose how they want to play. You can either solve the problem manually by clicking cells in the matrix or let the computer do it for you.

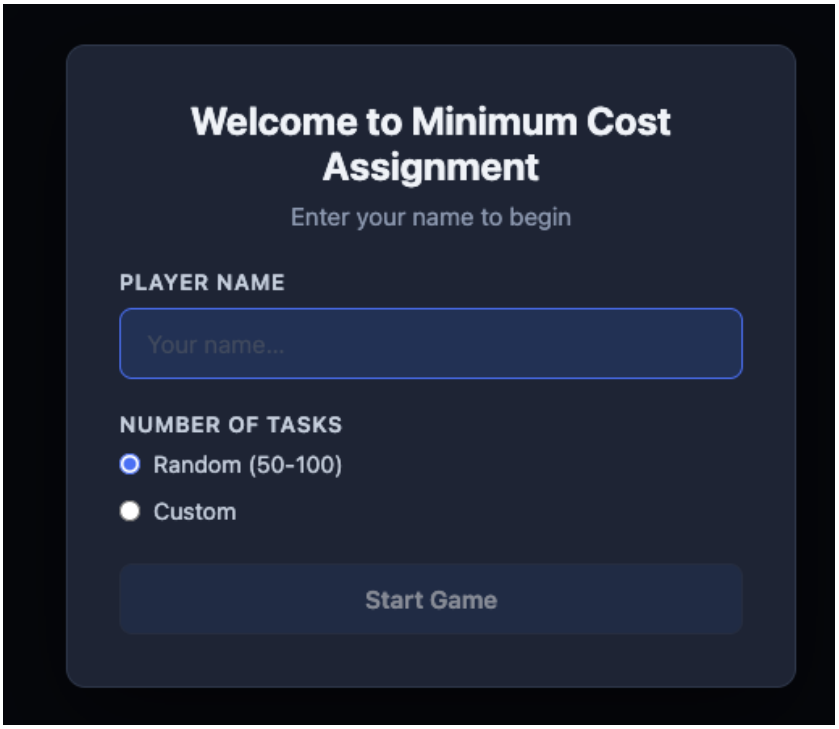

Figure 9: This shows where the user types their name before starting.

1.2.2. Validations and error handling

We added several checks to make the app stable:

- **Frontend (React)**: We made the UI so it prevents common mistakes. For example, you can't assign one worker to two different tasks by accident because the UI will automatically toggle the previous choice.
- **Backend (FastAPI)**: We use Pydantic models to check the data coming in. If the data is wrong or missing something, the backend sends an error instead of crashing. We also used try-except blocks in the database code to handle any SQL errors.

1.3. System Testing

1.3.1. Unit Testing

We used the pytest framework to test our code. We wrote several tests to make sure the algorithms work correctly. One important test was checking that the Hungarian algorithm always gives a cost that is less than or equal to the Greedy algorithm.

```python
import pytest
from algorithms.greedy import greedy_assignment
from algorithms.hungarian import hungarian_assignment


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------

def is_valid_assignment(assignment: list[int], n: int) -> bool:
    """Every employee has a unique task in range [0, n)."""
    return sorted(assignment) == list(range(n))


def is_valid_steps(steps: list[dict], n: int) -> bool:
    """Steps list covers all n employees with required keys."""
    if len(steps) != n:
        return False
    required = {"employee", "task
    return all(required.issubset(s.keys()) for s in steps)


# ---------------------------------------------------------------------------
# Greedy tests
# ---------------------------------------------------------------------------

class TestGreedyAssignment:

    def test_returns_valid_assignment_2x2(self):
        matrix = [[9, 2], [3, 8]]
        assignment, cost, steps = greedy_assignment(matrix)
        assert is_valid_assignment(assignment, 2)
        assert cost == sum(matrix[i][assignment[i]] for i in range(2))

    def test_returns_valid_assignment_3x3(self):
        matrix = [[4, 1, 3], [2, 0, 5], [3, 2, 2]]
        assignment, cost, steps = greedy_assignment(matrix)
        assert is_valid_assignment(assignment, 3)
        assert cost == sum(matrix[i][assignment[i]] for i in range(3))
```

(variable) s: dict[Unknown, Unknown]

Figure 10: Testing our Greedy logic with pytest.

```python
# ---------------------------------------------------------------------------
# Hungarian tests
# ---------------------------------------------------------------------------

class TestHungarianAssignment:

    def test_optimal_2x2(self):
        # Optimal: employee 0→task 1 (cost 2), employee 1→task 0 (cost 3) = 5
        matrix = [[9, 2], [3, 8]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 2)
        assert cost == 5

    def test_optimal_3x3_known_solution(self):
        # Optimal: 0→1 (1), 1→2 (1), 2→0 (1) = 3
        matrix = [[3, 1, 2], [2, 3, 1], [1, 2, 3]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 3)
        assert cost == 3

    def test_optimal_4x4_known(self):
        matrix = [
            [9,  2,  7,  8],
            [6,  4,  3,  7],
            [5,  8,  1,  8],
            [7,  6,  9,  4],
        ]
        # Brute-force verified optimal: 0→1 (2), 1→0 (6), 2→2 (1), 3→3 (4) = 13
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 4)
        assert cost == 13

    def test_returns_valid_assignment_single(self):
        matrix = [[99]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert assignment == [0]
        assert cost == 99

    def test_all_same_cost(self):
        matrix = [[7, 7, 7], [7, 7, 7], [7, 7, 7]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 3)
        assert cost == 21
```

Figure 11: Testing the Hungarian algorithm for correctness.

2. Appendix

GitHub link - https://github.com/adithyasean/game-project

All other deliverables are in this OneDrive link:

https://nibm-my.sharepoint.com/:f:/g/personal/cobsccomp242p-065_student_nibm_lk/IgC1z1sqJ0tgQ5aY-_paQP58ASlJvXpEXgEXOfWhQnmc9rQ?e=xGp5Ho